

Database Transactions and ACID Properties

What a transaction is, the states it moves through, and the four guarantees every DBMS makes about it.

Agenda

- 1 What is a database transaction?
- 2 Transaction processing: commit vs. abort
- 3 Transaction states — the full state diagram
- 4 The four ACID properties
- 5 Four worked ACID violations: what breaks without each property
- 6 Transaction management in the DBMS

DEFINITION

What Is a Database Transaction?

A **transaction** is a logical unit of work — one or more read/write operations that must be treated as a single, indivisible action. Either every operation completes and is recorded permanently, or none of them are.

There is no valid outcome where the database ends up “half-updated.”

PKR 10,000 TRANSFER: A101 → A205

```
BEGIN TRANSACTION;
```

```
UPDATE Account  
SET balance = balance - 10000  
WHERE accountNo = 'A101';
```

```
UPDATE Account  
SET balance = balance + 10000  
WHERE accountNo = 'A205';
```

```
COMMIT;
```

Transaction Processing

Transaction processing guarantees that each transaction's effect on the database is exactly as if it had run alone — even with hundreds of others executing at the same instant, and even if hardware fails mid-transaction.

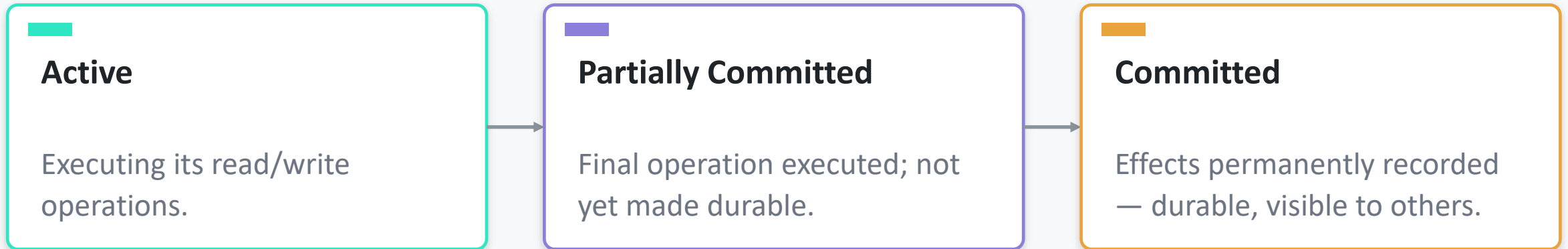
● Commit

Every operation succeeded. The DBMS makes all of the transaction's changes permanent.

● Abort (Rollback)

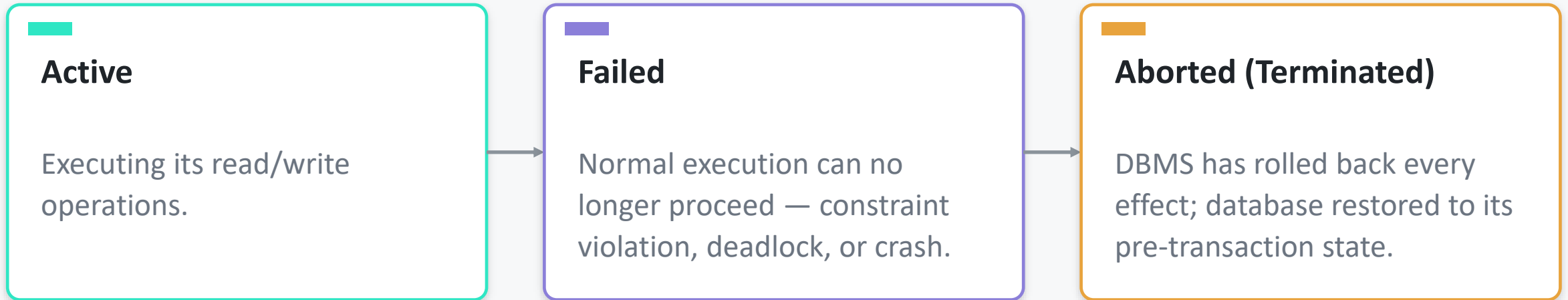
Something prevented completion. The DBMS undoes any partial changes, returning the database to its pre-transaction state.

Transaction States — The Successful Path



Committed and Aborted are always final. Once a transaction commits, nothing — not even a later crash — is allowed to undo it. That guarantee is Durability.

Transaction States — The Failure Path



Note: Partially Committed can also transition to Failed — if the DBMS discovers, while finalizing, that it cannot guarantee durability, it moves to Failed and then Aborted instead.

The ACID Properties

● Atomicity

All of a transaction's operations happen, or none do — no partial effect is ever left behind.

● Consistency

A transaction takes the database from one valid state to another, never violating an integrity constraint.

● Isolation

Concurrent transactions don't see each other's incomplete, in-progress effects.

● Durability

Once committed, a transaction's effects survive any later crash.

Atomicity and Consistency

Atomicity

The “all-or-nothing” property: a transaction's operations are treated as a single, indivisible unit. If any operation cannot complete, the DBMS undoes every operation that did already run — as if the transaction never started.

Consistency

A transaction, run to completion, takes the database from one valid state to another — never violating a domain, entity, referential, or general constraint. A DBMS enforces the constraints it knows about; encoding the business rule itself (e.g. “a transfer must debit exactly what it credits”) is the application's job.

Isolation and Durability

Isolation

Concurrently executing transactions do not interfere with one another — each behaves as though it were the only one running. Without isolation, one transaction could read another's half-finished work. This is a large enough topic to get its own full lecture next.

Durability

Once a transaction commits, its effects are permanent — they survive any subsequent failure. Delivered by writing a log of every change to stable storage before confirming the commit, so even a total loss of memory can be recovered from. The mechanics are Lecture 31's subject in full.

Isolation Violation: A Lost Update

A101 starts at 1000. T1 deposits 100; T2 deposits 50. Run serially, the correct final balance is 1150.

Time	T1	T2	balance
t1	T1 READ balance → 1000		1000
t2		T2 READ balance → 1000	1000
t3		T2 WRITE balance = $1000+50 = 1050$	1050
t4		T2 COMMIT	1050
t5	T1 WRITE balance = $1000+100 = 1100$		1100
t6	T1 COMMIT		1100

Final balance is 1100, not 1150 — T2's committed +50 deposit is silently lost. Full treatment, including how locking prevents this, is next lecture.

Atomicity Violation: The Balances, Snapshotted

PKR 10,000 transfer from A101 to A205, traced across a crash:

Scenario	A101	A205	Total
Before	50,000	20,000	70,000
During crash — no atomicity	40,000 (debited)	20,000 (credit never applied)	60,000 — PKR 10,000 missing
After recovery — with atomicity	50,000 (debit undone)	20,000 (unchanged)	70,000

Without atomicity, PKR 10,000 simply vanishes from the books. With it, the recovery manager undoes the incomplete debit on restart — the ledger is exactly as if the transfer never happened, and the customer can safely retry.

Consistency Violation: An Overdraft Rejected

Account A101 has balance 5,000, with a **CHECK (balance >= 0)** constraint declared on the column. A withdrawal runs **UPDATE Account SET balance = balance - 8000 WHERE accountNo = 'A101'** inside BEGIN...COMMIT — which would set balance to -3,000.

Without Consistency Enforcement

The UPDATE silently sets balance to -3,000 — a value with no valid real-world meaning for an account. Every later calculation and report built on that balance is now corrupted.

With Consistency Enforcement

The DBMS evaluates the CHECK constraint before the change stands. The constraint fails, the statement is rejected, and the transaction aborts — A101's balance remains 5,000. The application must handle 'insufficient funds' explicitly.

5,000 - 8,000 = -3,000. The CHECK (balance >= 0) constraint catches this before COMMIT — the UPDATE is rejected and A101 stays at 5,000.

Consistency Is a Joint Responsibility

Suppose a bug debits A101 by 10,000 but credits A205 by only 9,000. **Neither UPDATE violates any schema constraint** — both balances stay non-negative, no foreign key breaks. The DBMS has no way to know a debit must equal a credit, because that rule was never declared to it.

Consistency as the DBMS enforces it is necessary but not sufficient — this is not a bug Atomicity, Isolation, or Durability would ever catch.

A101 → A205 BUGGY TRANSFER

```
BEGIN TRANSACTION;
```

```
    UPDATE Account  
    SET balance = balance - 10000  
    WHERE accountNo = 'A101';
```

```
    UPDATE Account  
    SET balance = balance + 9000  
    WHERE accountNo = 'A205';
```

```
COMMIT;    -- PKR 1,000 lost
```

Durability Violation: A Crash One Millisecond After Commit

T1 (the 50,000 / 20,000 → 10,000 transfer) reaches COMMIT; the log record for both writes is flushed to stable storage and the DBMS acknowledges the commit. One millisecond later, the system crashes — before the in-memory data pages themselves are fully flushed to disk.

● Without Durability

On restart, the data files still show the pre-transfer balances — the 40,000 debit may or may not have reached disk, and the 20,000 → 30,000 credit never did. The committed transfer the customer was told succeeded has silently reverted.

● With Durability (WAL + Redo)

Because the log record reached stable storage before the commit was acknowledged, recovery replays it on restart and reapplies both writes. The committed transfer is guaranteed present no matter when the crash happened. Full mechanics: Lecture 31.

Durability is about WHEN data becomes safe, not just THAT it's saved: once the DBMS says 'committed,' the log — not yet the data pages — already holds everything recovery needs to redo it.

The Bank Transfer, Traced

Outcome 1 — both statements succeed

Step	Operation	State
1	BEGIN TRANSACTION	Active
2	Debit A101 by 10,000	Active
3	Credit A205 by 10,000	Active
4	Final operation done	Partially Committed
5	Log flushed to disk	Committed

Outcome 2 — crash after debit, before credit

Step	Operation	State
1	BEGIN TRANSACTION	Active
2	Debit A101 by 10,000	Active
3	(system crash)	Failed
4	Recovery manager undoes step 2	Aborted
5	A101 restored to original balance	(over)

What Atomicity Prevents

● Without Atomicity

Outcome 2 would leave A101 debited and A205 never credited — PKR 10,000 simply gone, with no record of where it went.

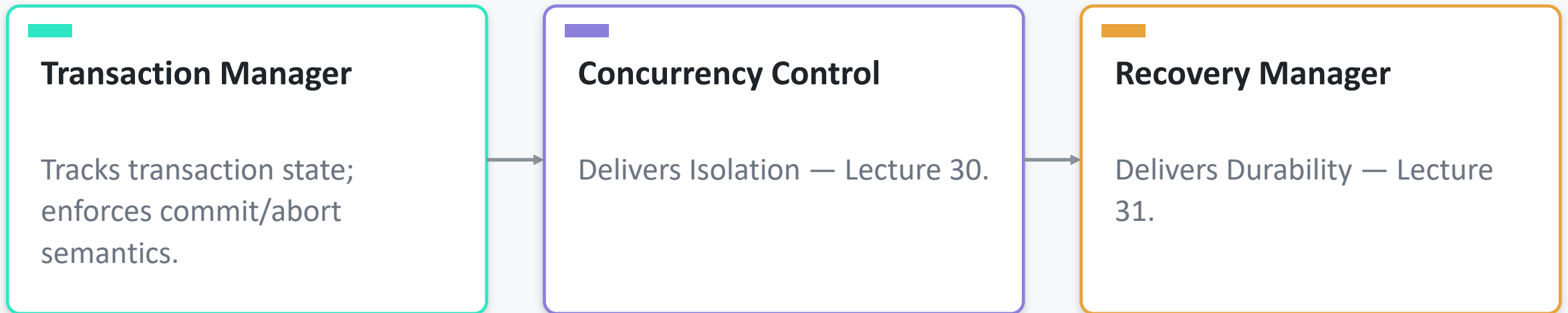
● With Atomicity

The recovery manager detects the incomplete transaction on restart, reverses the debit using the log — the books are exactly as if the transfer never happened.

Isolation without atomicity would still be broken: even perfect isolation between transactions says nothing about whether this transfer itself gets fully applied or fully reversed. All four ACID properties are independently necessary; none can substitute for another.

Transaction Management in the DBMS

The DBMS subsystem that tracks every transaction's state, enforces atomicity and consistency on commit or abort, and coordinates with two supporting subsystems.



All Four Properties, Side by Side

● Atomicity

Without it: PKR 10,000 vanishes between the debit and the credit — gone, with no record.

● Consistency

Without it: a balance goes to −3,000, or a debit/credit mismatch slips through with no error anywhere.

● Isolation

Without it: T2's committed +50 deposit is silently lost — the balance lands on 1100, not 1150.

● Durability

Without it: a committed transfer reverts after a crash — even though the customer was told it succeeded.

Key Takeaways

- A transaction is a logical unit of work — one or more read/write operations that must succeed or fail as a whole.
- A transaction moves through well-defined states: Active, Partially Committed, and either Committed or (via Failed) Aborted.
- ACID — Atomicity, Consistency, Isolation, Durability — is the precise definition of a correctly executed transaction.
- The bank-transfer trace showed what atomicity prevents: without it, a crash mid-transfer silently destroys money.
- Transaction management coordinates concurrency control and the recovery manager to deliver the full ACID guarantee.
- All four ACID properties were demonstrated with concrete violation scenarios, not just defined.